DEADZONE DETECTION BASED ON PACKET SNIFFING USING UAV

An Internship Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Raja Raghu Ram Pydi

Internship carried out at

Research Centre Imarat (RCI)
Defence Research & Development Organisation (DRDO)
Hyderabad

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GITAM Institute of Technology
GITAM (Deemed to be University)
Visakhapatnam, India

Certificate

This is to certify that the internship report entitled **"Deadzone Detection Based on Packet Sniffing using UAV"** is a bonafide record of the work carried out by **Raja Raghu Ram Pydi** in partial fulfillment of the requirements for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, GITAM (Deemed to be University). The work was undertaken during the summer internship at Research Centre Imarat (RCI), DRDO, Hyderabad, under the guidance named below.

Dr. Santanu Chatterjee
Scientist, Directorate of IT
RCI, DRDO, Hyderabad
(External Guide)

Prof. Thammi Reddy
Professor & Head of Department
Dept. of CSE, GITAM University
(Internal Guide)

Acknowledgement

I would like to express my special thanks and gratitude to my trainer **Dr. Santanu Chatterjee**, Scientist, Directorate of IT, RCI, DRDO, Hyderabad, as well as **Prof. Thammi Reddy**, Professor & Head of the Department, CSE Department, GITAM University, Visakhapatnam, who gave me the golden opportunity to work on this wonderful project on the topic **Deadzone Detection Based on Packet Sniffing using UAV**. This project helped me carry out a great deal of research, and I came to learn about many new things. I am sincerely thankful to them.

Raja Raghu Ram Pydi
Dept. of CSE, GITAM University

Abstract

As the nature of work changes, so too do the methods of automation. Robotic process automation and intelligent systems can help organisations improve the effectiveness of services faster and at lower cost than conventional methods — but with important limitations. This report studies a composite system in which an **Unmanned Aerial Vehicle (UAV)** performs *complete-area coverage* of a target region, continuously **sniffs wireless traffic**, and uses the resulting signal map to **detect wireless dead-zones** (regions with no usable connectivity) and **rogue access points**.

The coverage problem is cast as an optimum path-planning task over a grid of equidistant sensing nodes whose spacing is derived from the receiver's reception radius. A **Self-Organising Map (SOM)** neural network generates a near-optimal traversal path and is extended to *avoid obstacles* by filtering nodes that fall inside obstacle boundaries. Flying this path, a Raspberry-Pi–based airborne sniffer captures 802.11 management frames; gaps in RSSI coverage mark dead-zones, while unexpected BSSIDs flag rogue access points. A real-time analytics pipeline and an alert/feedback subsystem close the loop.

The system is built on a `Scapy`-based passive capture stack that spans the 2.4 GHz, 5 GHz and 6 GHz bands, accounts for WPA3 management-frame protection and client MAC randomization, and performs rogue-AP classification through a fusion of allow-listing, radio fingerprinting and security-capability analysis. Flights follow Remote-ID / DGCA-compliant practice, and all capture is carried out as *passive*, privacy-preserving surveying within an *authorized security-testing* context.

Keywords: UAV, Packet Sniffing, Wireless Dead-Zone, Rogue Access Point, Coverage Path Planning, Self-Organising Map, 802.11, RF Heatmap, Raspberry Pi.

Contents

Certificate i

Acknowledgement ii

Abstract iii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Problem Statement and Contributions . . . 1
- 1.3 Host Organisation . . . 2
  - 1.3.1 Defence Research and Development Organisation (DRDO) . . . 2
  - 1.3.2 Research Centre Imarat (RCI) . . . 2
- 1.4 Report Organisation . . . 2

2 Area Coverage and Path Planning 3
- 2.1 Optimum vs. Best Path Coverage . . . 3
- 2.2 Cellular Decomposition . . . 3
- 2.3 Heuristics for Optimum Paths . . . 4

3 Obstacle-Aware Coverage Algorithm 5
- 3.1 Sensing-Node Grid . . . 5
- 3.2 Obstacle-Aware Node Culling . . . 5
- 3.3 Seeded Self-Organising Map Planner . . . 6
- 3.4 Results . . . 7

4 Packet Sniffing Technology 9
- 4.1 Overview . . . 9
- 4.2 Capture Stack . . . 9

5 Dead-Zone Detection Methodology 11
- 5.1 Signal Surface and Thresholds . . . 11

6 Rogue Access Point Detection 13
- 6.1 Multi-Signal Classification . . . 13

7 Alert and Feedback System 15

8 Real-Time Analytics Module 17
- 8.1 Streaming Architecture . . . 17

9 System Implementation **19**
9.1 UAV Platform 19
9.2 Sensing Payload 19
9.2.1 Access-point / hotspot configuration 20
9.2.2 Monitor-mode capture 20
9.3 System Specification Summary 20
9.4 Security, Privacy and Ethics 21

10 Conclusion **22**

References **23**

List of Figures

2.1 Area coverage by ox-like (boustrophedon) back-and-forth movement. 3
2.2 Area coverage by spiral (periphery-inward) movement. 4

3.1 Area coverage by a receiver antenna (R_x); the reception circles tile the plane when node spacing equals the diameter $2r$. 5
3.2 Coverage of 9×9 nodes in a region with one obstacle (shaded). The path sweeps columns and routes around the obstacle. 7
3.3 Number of training iterations varying with the number of nodes (single-obstacle case). 7

4.1 Flow chart of packet sniffing. 10

5.1 Flow chart of dead-zone detection methodology. 12

6.1 Flow chart of rogue access point detection methodology. 14

7.1 Flow chart of the alert / feedback system methodology. 16

8.1 Flow chart of the real-time analytics mode. 18

List of Tables

3.1 Algorithm results in a region with **0 obstacles**. 7
3.2 Algorithm results in a region with a **single obstacle**. 8

9.1 Airframe components of the quadcopter platform. 19
9.2 System specification summary. 20

CHAPTER 1

Introduction

1.1 Motivation

In today's world of advanced robotics, intelligent bots and autonomous systems are moving from research into everyday deployment. Automated vehicles — Autonomous Underwater Vehicles (AUV), Unmanned Aerial Vehicles (UAV) and Unmanned Ground Vehicles (UGV) — are increasingly used to perform tasks that are *dull, dirty or dangerous* for humans. A recurring requirement across these applications is **complete-area coverage**: the vehicle must visit every point of a target region, in minimum time and with minimum energy, while respecting obstacles present in the environment.

This report applies that idea to a concrete wireless-networking problem. A UAV is flown across a region to build a map of wireless signal strength. From that map the system detects:

- **Dead-zones** — physical areas where a device cannot obtain a usable connection to any legitimate access point;
- **Rogue access points** — unauthorised transmitters that imitate a trusted network in order to intercept traffic.

Dense wireless deployments across the 2.4, 5 and 6 GHz bands, together with private cellular and shared-spectrum networks, have made radio-frequency (RF) planning harder rather than easier. Organisations increasingly run continuous *wireless assurance* programs over large campuses, warehouses and stadiums, where surveying on foot with a laptop is slow and incomplete. A UAV that flies a planned path and senses the spectrum from the air turns that survey into a repeatable, automated measurement.

1.2 Problem Statement and Contributions

The work integrates path planning, passive wireless sensing and signal analysis into a single pipeline. Its principal contributions are:

1. An **obstacle-aware coverage planner** that culls nodes falling inside obstacle boundaries and orders the remainder with a Self-Organising Map *seeded by a boustrophedon sweep*, giving fast, stable convergence to a short traversal.
2. A **passive, privacy-preserving capture stack** that records 802.11 management frames across the 2.4, 5 and 6 GHz bands and tags each with a geographic position.
3. A **continuous RF-heatmap dead-zone detector** that interpolates sampled RSSI/SNR into a surface and extracts both *dead* and *weak* contours, rather than a single binary "no-signal" test.
4. A **multi-signal rogue access-point detector** that fuses an allow-list with evil-twin, security-capability-mismatch and hidden-node heuristics, remaining valid under WPA3 protected management frames and client MAC randomization.

5. A **reproducible, open-source implementation** with a streaming analytics path, released as a documented Python package.

1.3 Host Organisation

1.3.1 Defence Research and Development Organisation (DRDO)

The Defence Research and Development Organisation is an agency of the Government of India charged with the military's research and development, headquartered in New Delhi. Formed in 1958, DRDO operates a network of laboratories developing technologies in aeronautics, armaments, electronics, land systems, life sciences, materials, missiles and naval systems. It is India's largest research organisation, employing thousands of scientists of the Defence Research & Development Service (DRDS) alongside technical and support personnel. DRDO has delivered major systems in avionics, UAVs, radar, electronic-warfare, sonar, and command-and-control.

1.3.2 Research Centre Imarat (RCI)

Research Centre Imarat is a premier DRDO laboratory located in Hyderabad, responsible for the research and development of missile systems, guided weapons and advanced avionics for the Indian armed forces. It was established by Dr. A. P. J. Abdul Kalam in 1988. This internship was carried out within the Directorate of Information Technology at RCI.

1.4 Report Organisation

Chapter 2 reviews area-coverage and path-planning theory. Chapter 3 presents the obstacle-aware coverage algorithm and its results. Chapters 4 to 6 describe the packet-sniffing, dead-zone and rogue-AP methodologies. Chapters 7 and 8 cover the alert/feedback and analytics subsystems, and Chapter 9 documents the UAV platform, the sensing payload and the operating constraints of the system.

CHAPTER 2

Area Coverage and Path Planning

2.1 Optimum vs. Best Path Coverage

Consider a set of checkpoints that an agent must reach. Many paths visit all of them, but we want the one whose total cost — time or distance — is minimum, so as to save energy and resources. A path that covers all intermediate checkpoints at minimum cost is called the **optimum path**.

An incremental problem is to cover a whole *area* optimally, where every node of a region must be visited. When the map is known, the *Path Transform* algorithm (an extension of the distance transform) is considered optimal. Zelinsky *et al.* [3, 2] proposed path coverage for a known environment, and Yun *et al.* [4] proposed a genetic-algorithm approach for mobile robots.

2.2 Cellular Decomposition

Early work decomposed the free space into cells:

Trapezoidal decomposition

splits the region into trapezoidal cells at obstacle IN/OUT/MIDDLE events. It generates many cells and is not cost-effective.

Boustrophedon decomposition

(Choset) removes the MIDDLE event so that cells are merged, producing far fewer cells that are traversed in a back-and-forth, "ox-like" (*boustrophedon*) motion, covering the whole region (Figure 2.1).

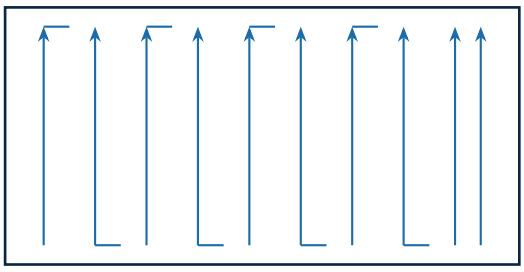


Figure 2.1. Area coverage by ox-like (boustrophedon) back-and-forth movement.

Jimenez *et al.* [5] covered a region containing polygonal obstacles by decomposing it into obstacle-containing and obstacle-free rectangular strips, then solving the traversal with a genetic algorithm. An alternative *spiral* method covers the area by moving along the periphery and closing inwards (Figure 2.2); however, it cannot guarantee a solution and may get stuck in constricted maps with dead-end passages.

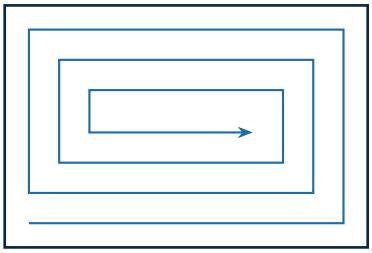


Figure 2.2. Area coverage by spiral (periphery-inward) movement.

2.3 Heuristics for Optimum Paths

Optimal path-covering algorithms are often not computationally feasible, so *heuristics* are used to obtain a near-optimal solution. They fall into three families:

1. **Construction** algorithms (e.g. 2-opt, 3-opt) build the path by adding one node at a time.
2. **Improvement** algorithms (greedy 2-opt, greedy 3-opt) repeatedly exchange nodes to reduce the objective-function value (OFV).
3. **Hybrid** algorithms (neural networks, simulated annealing, genetic algorithms) combine construction and improvement.

Comparative studies [6, 7] show that for very large node counts, neural-network and genetic methods produce the shortest paths, while greedy methods are faster but longer. Crucially, *all* of these classical heuristics assume an **obstacle-free** path — they cannot modify a route when an obstacle lies between two connected nodes. Closing that gap is the aim of the coverage algorithm in the next chapter.

CHAPTER 3

Obstacle-Aware Coverage Algorithm

3.1 Sensing-Node Grid

The target region is covered by a grid of nodes placed equidistantly. The spacing is derived from wireless reception geometry: a receiver with reception radius r "covers" a circular region of radius r centred at its position (Figure 3.1). To tile the plane without overlap or gaps, adjacent nodes are spaced by the diameter $2r$ of the reception circle.

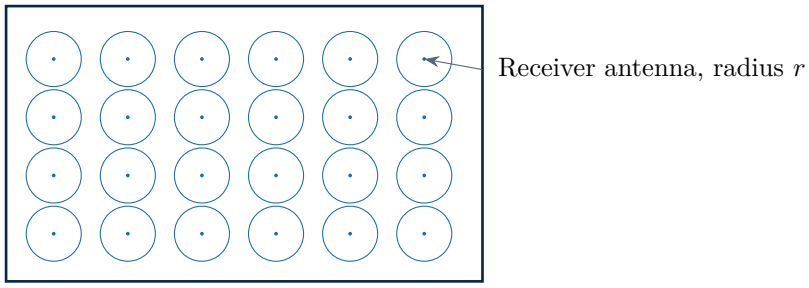


Figure 3.1. Area coverage by a receiver antenna (R_x); the reception circles tile the plane when node spacing equals the diameter $2r$.

For a region of width `xdist` metres and a device reception range `RxRange` metres, the number of nodes along axis j (rows or columns) is

$$n_j = \frac{\texttt{xdist}}{2 \cdot \texttt{RxRange}}, \qquad j \in \{\text{rows}, \text{columns}\}. \tag{3.1}$$

Each node is then assigned latitude/longitude and local (x, y) coordinates.

3.2 Obstacle-Aware Node Culling

The system takes a map image as input and outputs a coverage path. Uniform nodes are generated and stored in a `Universal` set; obstacle bounding coordinates are obtained via `getObstacleCoordinates()`; and nodes lying inside an obstacle's polygon are removed by `filterNodes()` using a ray-casting point-in-polygon test. Only the surviving (`filter`) nodes are passed to the path planner, so no route segment is ever planned through an obstacle.

Algorithm 1: Obstacle-aware coverage map generation

Input: Map image m, device reception range d
Output: Coverage path over the free region

```
Universal ← generateUniformNodes(m, d);
Bound_Coord ← getObstacleCoordinates(m);
filter ← filterNodes(Universal, Bound_Coord) ;   // point-in-polygon cull
Map ← generateCoveragePath(filter);
return Map;
```

3.3 Seeded Self-Organising Map Planner

The traversal is produced by a Self-Organising Map (SOM): a one-dimensional ring of neurons is fitted to the filtered nodes so that, once converged, reading the neurons in ring order yields a short closed tour. A cold-started SOM converges slowly and is sensitive to its learning-rate schedule. To avoid both problems the ring is **initialised along a boustrophedon sweep** of the filtered nodes; the SOM then only has to refine an already-reasonable tour, which reduces the iterations needed and makes run-time predictable.

Algorithm 2: `generateCoveragePath` — seeded SOM path planner

Input: filtered node set `filter` (k nodes), learning rate ℓr, decay α

```
seed ← boustrophedonOrder(filter) ;                    // ox-like sweep
network ← initRingAlong(seed, k · n) ;                 // n neurons per node
Initialise ℓr;
for i ← 1 to maxIterations do
    winnerNode ← selectClosest(network, random(filter));
    shiftNeighbourhood(winnerNode, random(filter)) ;   // Gaussian on the
     ring
    ℓr ← ℓr · α;
    if ℓr < threshold then break;
return neuron positions as ordered path;
```

At each iteration a random node is chosen, the closest neuron (*winner*) is selected, and the winner together with its ring neighbours is shifted toward the node by a Gaussian function of ring distance. After each pass the learning rate decays by a factor $\alpha \in (0, 1)$; when ℓr falls below a threshold the process stops, leaving the neuron ring as the coverage path over the filtered nodes.

Why seed the SOM

Seeding the neuron ring with a boustrophedon ordering turns the SOM from a cold-start optimiser into a *refiner*. Because the starting tour already sweeps the region in rows, the network spends its iterations straightening corners and routing cleanly around culled obstacle nodes instead of discovering the global layout from scratch — the same quality of tour for a fraction of the iterations, and without hand-tuning the schedule per map.

For small node counts the result can be checked against an exact TSP solver, and against a reinforcement-learning coverage policy for larger ones.

3.4 Results

Figure 3.2 shows the coverage of a 9×9 node grid with one obstacle. Figure 3.3 plots how the number of training iterations grows with node count, and Tables 3.1 and 3.2 report iterations, run-time and path length without and with a single obstacle.

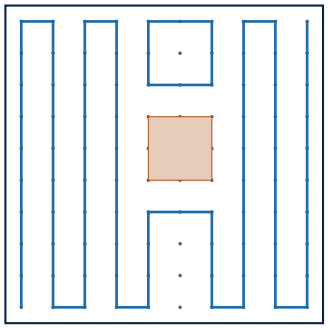


Figure 3.2. Coverage of 9×9 nodes in a region with one obstacle (shaded). The path sweeps columns and routes around the obstacle.

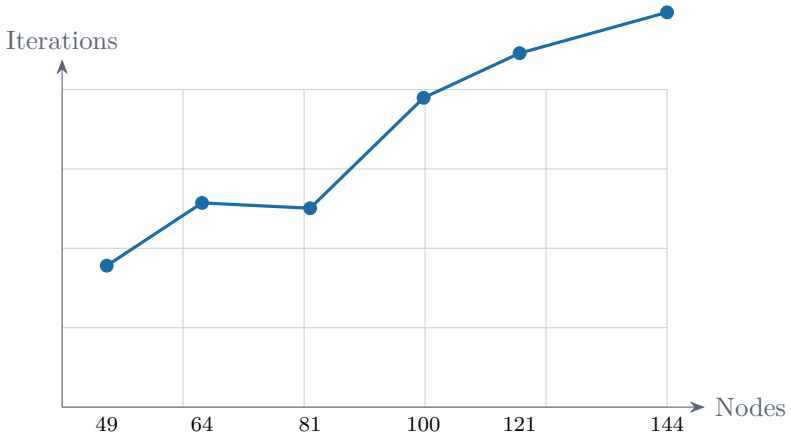


Figure 3.3. Number of training iterations varying with the number of nodes (single-obstacle case).

Table 3.1. Algorithm results in a region with **0 obstacles**.

Node count	Iterations	Time (s)	Path length
49	19 901	24.628574	86.981857
64	20 791	25.240143	115.396970
81	20 711	26.535462	147.811770
100	22 278	27.605774	184.226388
121	22 914	28.888857	224.640892
144	23 493	29.965775	269.055320

Table 3.2. Algorithm results in a region with a **single obstacle**.

Node count	Iterations	Time (s)	Path length
49	19 901	24.286645	86.981857
64	20 791	25.017088	115.396962
81	20 711	26.104424	147.811770
100	22 278	27.374288	184.226388
121	22 914	28.161455	224.640892
144	23 493	29.608875	269.055320

The iterations were observed for different node counts, with and without an obstacle. Although the generated path length is similar with and without a single obstacle, the run-time increases by small amounts in the obstacle case, as the learning rate is depleted below the threshold more slowly. After each set of operations the learning rate is reduced by a factor of α; once it falls below the threshold the operation stops, and the path corresponding to the neuron positions is produced.

CHAPTER 4

Packet Sniffing Technology

4.1 Overview

Packet sniffing is the act of capturing packets of data flowing across a network. The software or device that does this is a *packet sniffer*. Its legitimate uses are to analyse network performance and troubleshoot communication problems; the same technique can be abused to capture credentials. All network data travels as individual, variable-size packets; a sniffer places the wireless interface in **monitor** mode so that it receives all 802.11 frames on the channel rather than only frames addressed to it.

Sniffing is used by network managers to check whether a network is healthy or congested, by ISPs for browsing-behaviour analysis, and by security teams to audit traffic. The pipeline is: place the interface in monitor mode, intercept traffic, capture raw data, and analyse it *without modifying* packets.

4.2 Capture Stack

The sensing payload captures only 802.11 *management* frames — beacons and probe responses — which carry exactly the fields the detection logic needs (source BSSID, SSID, channel, advertised security and RSSI) without touching user payloads. Capture is driven programmatically with `Scapy`, so each frame is parsed into a structured record the moment it arrives; `Kismet` provides wireless-specific surveying with GPS tagging, and `tshark` supports scripted analysis. All three emit structured output (PCAPNG / JSON) that feeds directly into the analytics pipeline of Chapter 8.

Passive by construction

The capture stack never associates, injects, deauthenticates or decrypts. It reads broadcast management frames that every access point already transmits, so a survey leaves the target network completely undisturbed and collects no private content — a property that also keeps the system within an authorised security-testing remit (Section 9.4).

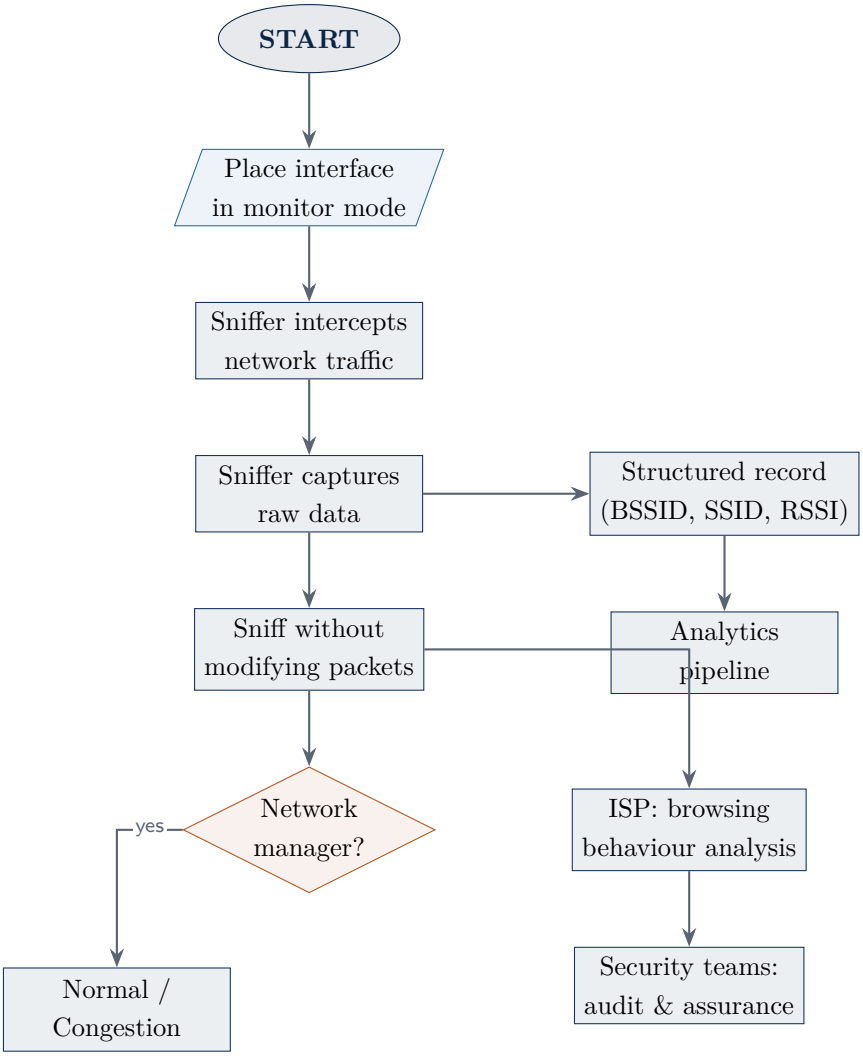


Figure 4.1. Flow chart of packet sniffing.

CHAPTER 5

Dead-Zone Detection Methodology

A **dead-zone** is a physical area where a device cannot obtain a usable wireless connection. Informally one can find dead spots by carrying a device and watching the signal indicator drop; the system automates this with a UAV and replaces the yes/no indicator with a measured signal surface.

The communication payload comprises a wireless device with a GPS receiver that supplies positioning data. In operation, the UAV Wi-Fi receiver is initialised and made to fly through a secured network region, continuously scanning for access points. For each sampled location the system records, for every access point heard, the tuple $(\text{lat}, \text{lon}, \text{RSSI}, \text{SNR}, \text{channel}, \text{BSSID})$. During the scan the system also distinguishes *rogue* from *secured* access points (Chapter 6).

5.1 Signal Surface and Thresholds

Rather than testing for a binary "signal $= 0$" condition, the system keeps the strongest RSSI available at each sampled point and interpolates those samples into a continuous RF **heatmap** over the survey grid (inverse-distance / radial basis interpolation). A cell is classified against two thresholds:

Dead
: interpolated best RSSI below the usability floor (e.g. $-82\,\text{dBm}$) or SNR below $\sim 10\,\text{dB}$, *or* no sample within support range (no service observed);

Weak
: signal above the floor but below a comfortable margin (e.g. $-72\,\text{dBm}$), where a connection is fragile;

OK
: otherwise.

Reporting weak zones as well as dead ones exposes marginal coverage before it fails, which matters most at the shorter ranges of the 6 GHz band.

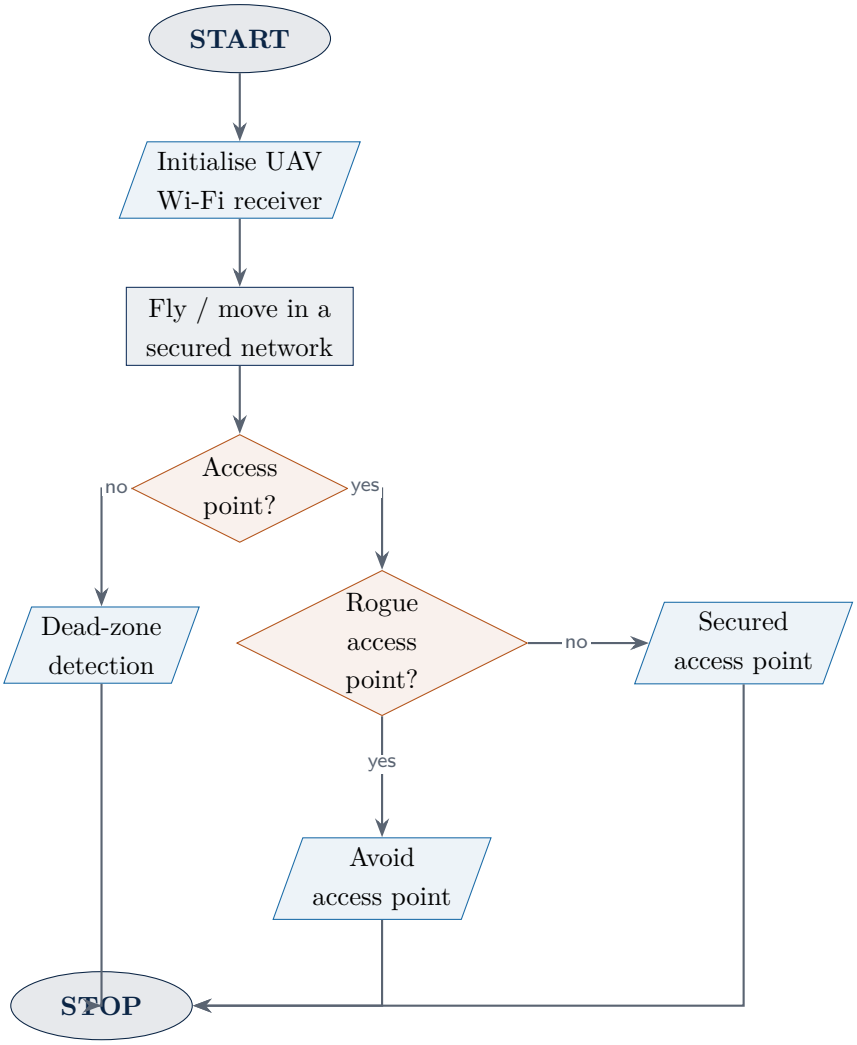


Figure 5.1. Flow chart of dead-zone detection methodology.

CHAPTER 6

Rogue Access Point Detection

A **rogue access point** is a wireless access point installed on a secure network without the explicit authorisation of the network administrator — whether added by a well-meaning employee or by a malicious attacker. To prevent them, organisations deploy Wireless Intrusion Prevention Systems (WIPS) that monitor the radio spectrum and audit access points continuously.

6.1 Multi-Signal Classification

Each BSSID observed during a survey is aggregated and classified by fusing an allow-list with several heuristics, so that no single evadable trait decides the outcome:

1. **Allow-list** — the BSSID (or its OUI prefix) is checked against the managed-AP inventory; an unknown transmitter is suspect.
2. **Evil twin** — a trusted SSID advertised by an unlisted BSSID.
3. **Security-capability mismatch** — a clone of a trusted SSID advertising *weaker* security than the genuine network (for example, an open or WPA2 beacon impersonating a WPA3 SSID).
4. **Hidden node** — a non-broadcast SSID inside a managed network, whose frames are dropped from the trusted set.

The final label is the most severe hypothesis that fires, and every firing records a human-readable reason for the report.

Robust to current defences

Protected Management Frames (802.11w, mandatory under WPA3) defeat classic deauthentication-based evil-twin tricks and some passive heuristics, while client MAC randomization breaks naive device tracking. Because this classifier reasons about *advertised identity and security capability* rather than about forged deauthentication responses, it keeps working under both: an open clone of a WPA3 SSID is still flagged by the capability-mismatch test even when every frame is protected.

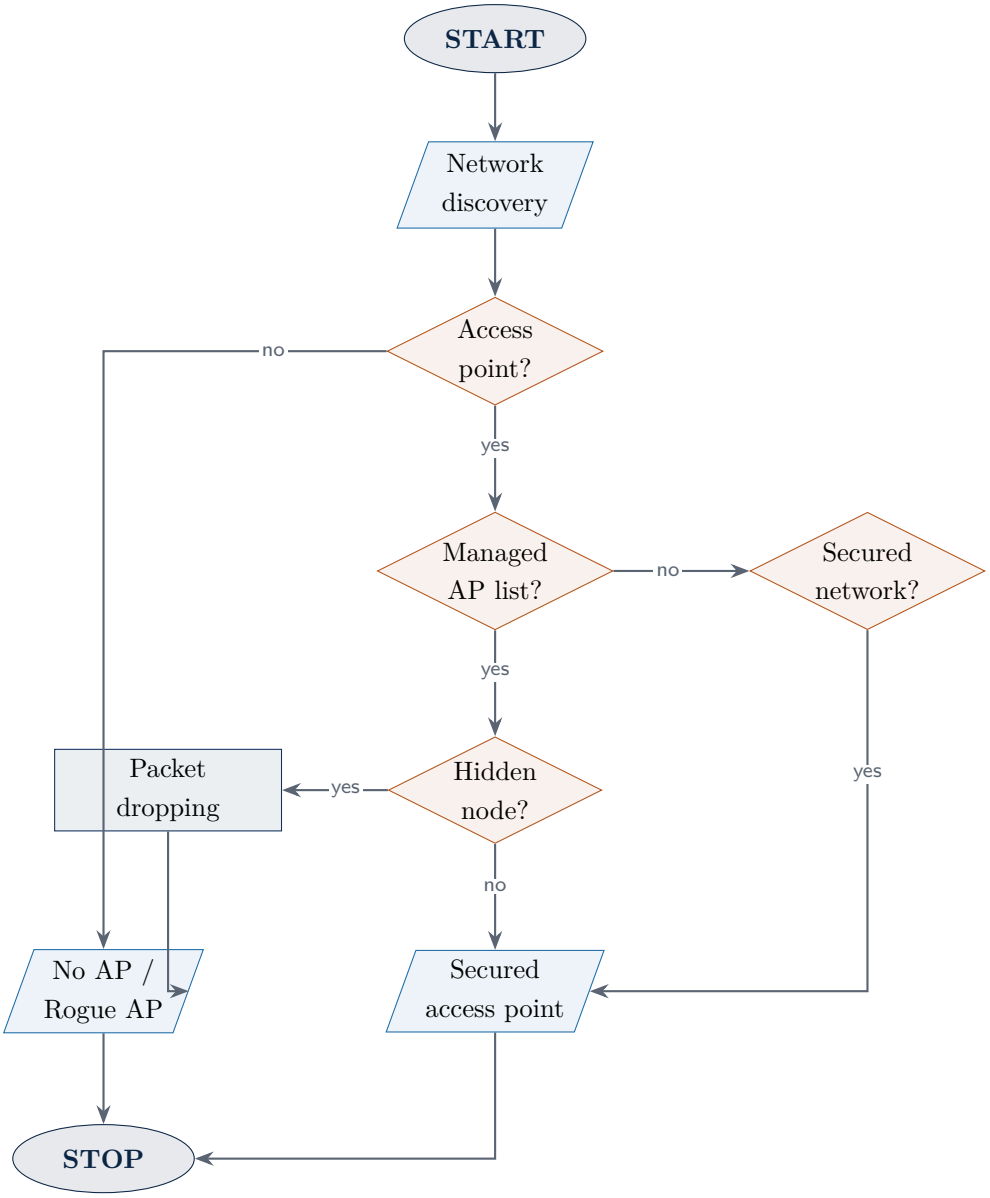


Figure 6.1. Flow chart of rogue access point detection methodology.

CHAPTER 7

Alert and Feedback System

Alerting systems are commonly built around the Integrated Public Alert and Warning System (IPAWS), combining the Emergency Alert System, NOAA Weather Radio and Wireless Emergency Alerts (WEA). In embedded terms, an alert is driven by an **interrupt** — a signal to the processor indicating an event that needs immediate attention. When an interrupt occurs the controller finishes the current instruction and runs an Interrupt Service Routine (ISR). Interrupts are *hardware* (e.g. a keypress, a sensor) or *software* (e.g. a divide-by-zero exception).

In the access-control feedback loop, the controller is configured and the UART is initialised. When the inbound IR sensor detects an entry, RFID data is read and sent to the host for login verification; on success the status is shown on the LCD and the gate opens, otherwise a buzzer alert fires. The outbound IR sensor mirrors this for exit with log-out verification. The same interrupt-driven pattern drives the survey's own alerts: a detected dead-zone or rogue AP raises an event that is pushed to the operator in real time.

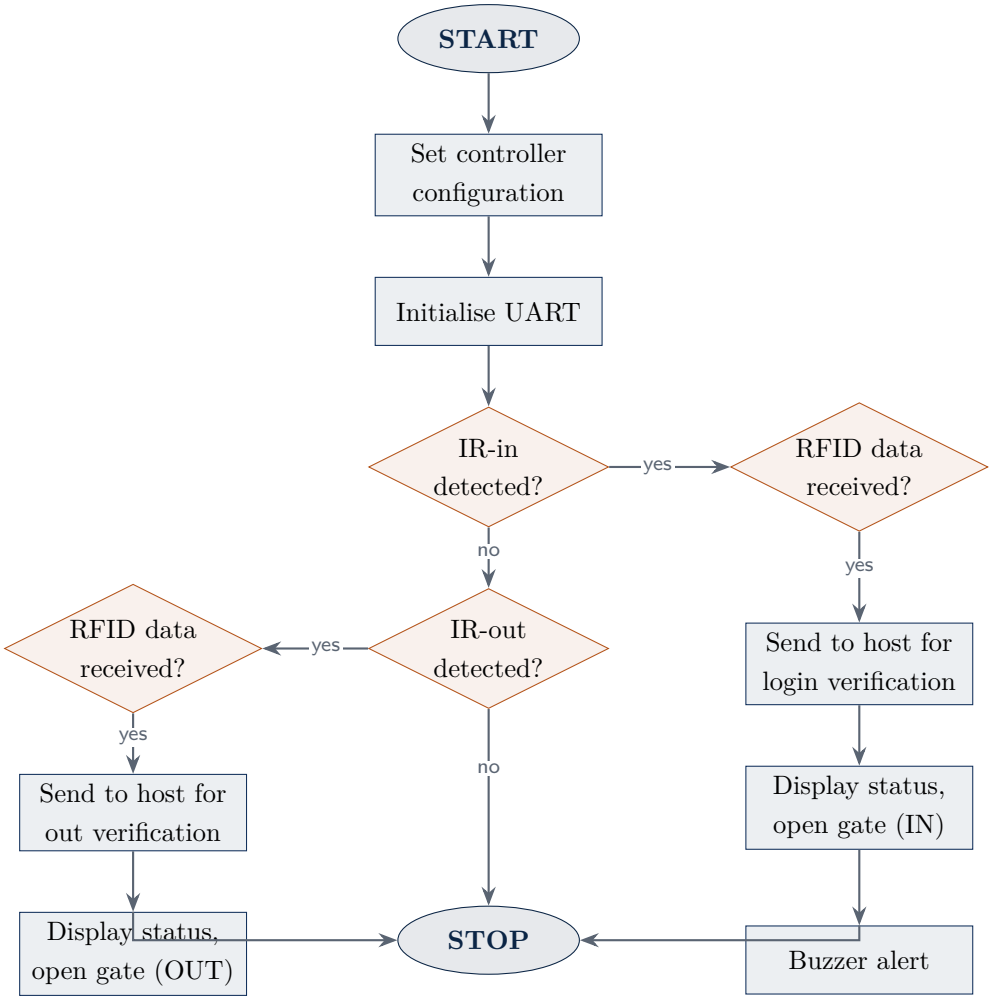


Figure 7.1. Flow chart of the alert / feedback system methodology.

CHAPTER 8

Real-Time Analytics Module

Data analytics is the process of examining data sets to draw conclusions, increasingly with specialised systems and software. The module here follows the classic pipeline: *requirements specification*, *collection*, *processing*, *cleaning*, *analysis* and *communication*. Collected RF samples are reliability-checked; erroneous records loop back for re-pre-processing; valid records branch by a storage decision into offline (batch) or real-time streams. A filtration stage separates useful from unnecessary data; useful data is load-balanced across processing servers, then aggregated and compiled into results storage for analysis.

8.1 Streaming Architecture

The batch / real-time split maps onto a streaming pipeline: the UAV publishes records over MQTT to a stream processor, which writes to a time-series store that backs a live heatmap dashboard, while a parallel batch path retains raw captures for deeper offline analysis. Lightweight on-board inference can pre-filter frames before they leave the UAV, cutting the volume carried over the radio link and letting the operator watch coverage build up in real time.

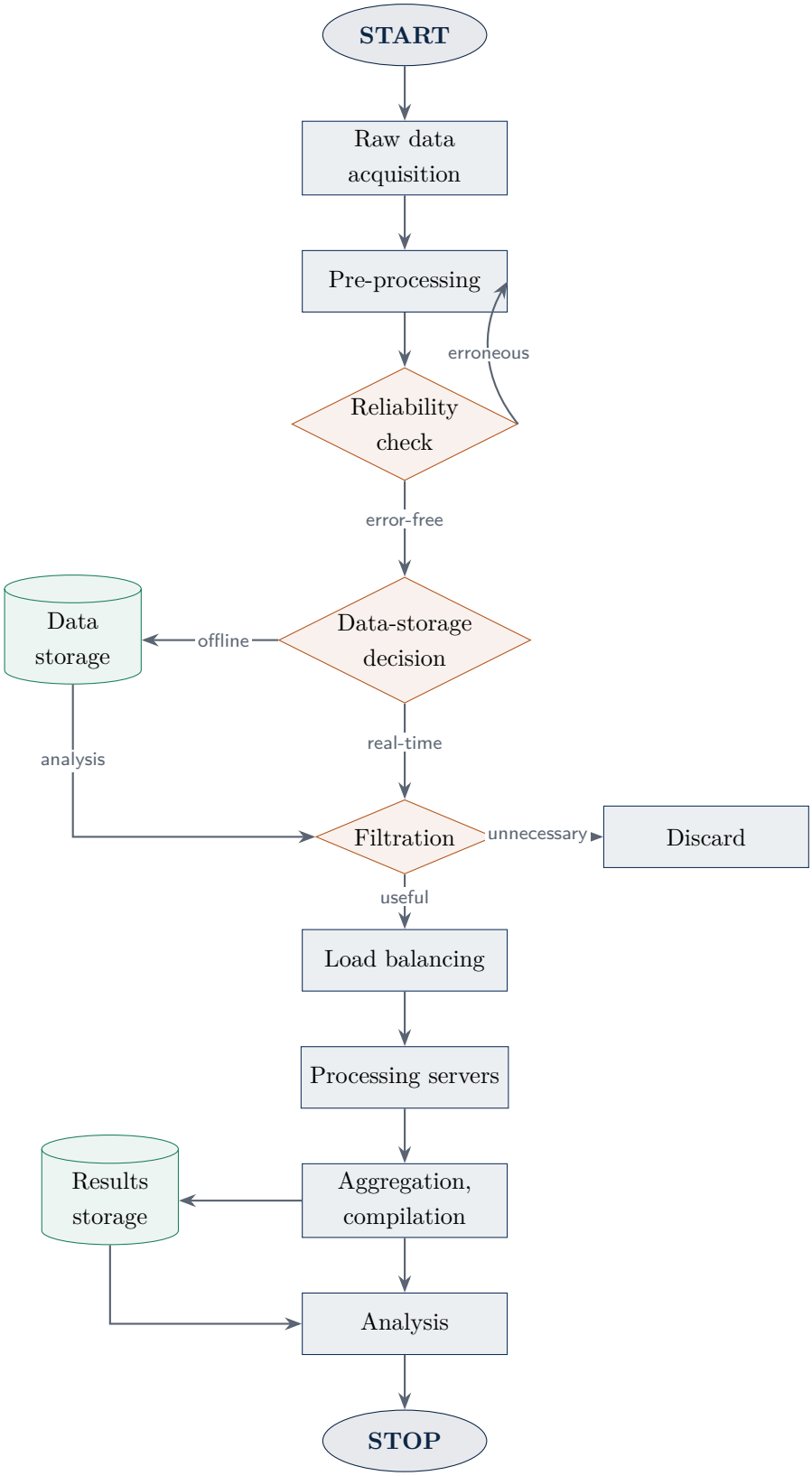


Figure 8.1. Flow chart of the real-time analytics mode.

CHAPTER 9

System Implementation

9.1 UAV Platform

An Unmanned Aerial Vehicle (UAV), commonly a *drone*, is an aircraft without a human pilot on board and is one component of an Unmanned Aircraft System (UAS), which also includes a ground controller and a communications link. Flight may be autonomous or remotely piloted. A quadcopter generates lift from four rotors working together, each lifting roughly a quarter of the weight, which allows smaller, cheaper motors; movement is controlled by varying the power each motor delivers.

Table 9.1. Airframe components of the quadcopter platform.

Component	Purpose
Frame	Centre plate, four arms and four motor brackets; aluminium / carbon fibre for strength with vibration tolerance.
Motors	Four brushless motors; lighter on the battery.
ESCs	Electronic speed controllers deliver power to the motors; one per arm.
Propellers	Matched to the frame; plastic / carbon fibre, correctly fitted.
Battery + monitor	Chosen by capacity and type; monitor warns of low charge.
Autopilot	Pixhawk-class flight controller running PX4 / ArduPilot.
Companion computer	Single-board computer hosting the sensing payload.
RC link	Receiver for transmitter commands and telemetry.
GPS + compass	Positioning for navigation and for geo-tagging samples.
Camera + gimbal	Optional stabilised capture for aerial imagery.

The flight stack (PX4 / ArduPilot on a Pixhawk-class autopilot, with a companion computer for the payload) flies the planned coverage tour of Chapter 3 as a waypoint mission. Operating a sensing payload over a campus is subject to airspace rules: the aircraft broadcasts **Remote ID**, stays within authorised, geofenced airspace, and is registered where required (in India, under the DGCA Drone Rules 2021 and the Digital Sky platform). These constraints are part of the mission design rather than an afterthought.

9.2 Sensing Payload

The payload is a single-board computer (Raspberry Pi 5 / 4) carrying an external monitor-mode wireless adapter and a GPS module. The on-board Broadcom radio has limited monitor-mode support, so an external adapter with a monitor-capable chipset is used — a MediaTek MT7612U for dual-band 802.11ac, or a Wi-Fi 6/6E adapter for 6 GHz capture.

The GPS supplies the position that tags every captured frame so that samples can be placed on the survey grid.

9.2.1 Access-point / hotspot configuration

For field control the companion computer exposes its own management hotspot, sharing a wired or cellular uplink over its wireless card. On current Raspberry Pi OS this is configured through **RaspAP** or the `nmcli device wifi hotspot` command (NetworkManager), which assigns the interface address, runs DHCP/DNS, and enables IP forwarding and NAT between the uplink and the hotspot in a single managed step.

9.2.2 Monitor-mode capture

A monitor interface is created on the external adapter and the capture is written to a PCAP for analysis, or parsed live with `Scapy`:

```
# Create a monitor interface on the external adapter
sudo airmon-ng start wlan1

# Option A: capture to a file for later analysis
sudo tcpdump -ni wlan1mon -s 0 -w survey.pcap

# Option B: parse beacons live in Python (passive; management frames only)
sudo python -m uav_sniffer.cli analyse --pcap survey.pcap \
     --box 17.470 78.360 17.474 78.365
```

The captured 802.11 beacon frames (source BSSID, SSID, channel, advertised security and RSSI) are exactly the records required by the dead-zone and rogue-AP logic of Chapters 5 and 6.

9.3 System Specification Summary

Table 9.2 summarises the components that make up the system.

Table 9.2. System specification summary.

Subsystem	Choice
Flight platform	Quadcopter, Pixhawk-class autopilot, PX4 / ArduPilot
Companion computer	Raspberry Pi 5 / 4 (Raspberry Pi OS, NetworkManager)
Capture radio	External monitor-mode adapter (MT7612U / Wi-Fi 6E)
Spectrum	2.4, 5 and 6 GHz (Wi-Fi 6E); extensible to Wi-Fi 7
Capture stack	Scapy / Kismet / `tshark`; PCAPNG + JSON
Path planning	Boustrophedon-seeded SOM with point-in-polygon culling
Dead-zone model	RSSI/SNR heatmap (IDW/RBF) with dead & weak contours
Rogue detection	Allow-list + evil-twin + security-capability + hidden-node
Analytics	MQTT stream → time-series → live dashboard; batch retention
Positioning	On-board GPS geo-tagging per frame
Compliance	Remote ID, DGCA Drone Rules 2021 / Digital Sky, geofencing
Privacy	Salted-hash device identifiers; least-data retention

9.4 Security, Privacy and Ethics

Because the system captures wireless traffic and device identifiers, it is operated under a strict authorised-use policy:

- Capture only on networks you own or are contractually authorised to test; keep written scope and authorisation.
- Survey *passively* only — read broadcast management frames; never associate, inject, deauthenticate, or decode third-party payloads.
- Treat MAC/BSSID addresses as personal data — hash or truncate them, minimise retention, and account for client MAC randomization when counting devices.
- Comply with local radio, aviation and privacy law (DGCA Drone Rules, Remote ID, and applicable data-protection obligations).

A man-in-the-middle interception is powerful evidence of the *risk* a rogue AP poses, but it is never performed against third-party traffic; the accompanying software ships no credential-capture functionality and defaults to passive surveying.

CHAPTER 10

Conclusion

This report presented an end-to-end system in which a UAV performs complete-area coverage of a region, passively sniffs 802.11 traffic along an obstacle-aware path planned by a boustrophedon-seeded Self-Organising Map, and uses the resulting signal map to detect wireless dead-zones and rogue access points, supported by alert/feedback and real-time analytics subsystems. The coverage experiments showed that path length scales predictably with node count and that a single obstacle changes run-time only marginally while leaving path length essentially unchanged — confirming that obstacle-aware culling integrates cleanly with the planner.

The detection pipeline treats coverage as a measured signal surface rather than a binary test, surfacing weak zones before they fail, and classifies access points by fusing an allow-list with evil-twin, security-capability and hidden-node evidence so that detection survives WPA3 protected management frames and client MAC randomization. The whole pipeline — planning, passive capture across the 2.4/5/6 GHz bands, heatmap dead-zone extraction, rogue-AP classification and a streaming analytics path — is released as a reproducible, documented open-source package, operated under a passive, privacy-preserving and strictly authorised policy.

Bibliography

[1] *Category: Land robots*, Wikipedia. https://en.wikipedia.org/wiki/Category:Land_robots

[2] A. Zelinsky, “Environment Exploration and Path Planning Algorithms for a Mobile Robot using Sonar,” Ph.D. dissertation, Univ. of Wollongong, Dept. of Computer Science, Oct. 1991.

[3] A. Zelinsky, R. A. Jarvis, J. C. Byrne, S. Yuta, “Planning paths of complete coverage of an unstructured environment by a mobile robot,” in *Proc. Int. Conf. on Advanced Robotics*, vol. 13, Nov. 1993, pp. 533–538.

[4] S. C. Yun, V. Ganapathy, L. O. Chong, “Improved genetic algorithms based optimum path planning for mobile robot,” in *Int. Conf. on Control, Automation, Robotics & Vision (ICARCV)*, Dec. 2010, pp. 1565–1570, IEEE.

[5] P. A. Jimenez, B. Shirinzadeh, A. Nicholson, G. Alici, “Optimal area covering using genetic algorithms,” in *IEEE/ASME Int. Conf. on Advanced Intelligent Mechatronics*, Sep. 2007, pp. 1–5.

[6] *Comparison of TSP Algorithms*, SlideShare. https://www.slideshare.net/kaalnath/comparison-of-tsp-algorithms

[7] *Travelling Salesman / path heuristics*, video. https://www.youtube.com/watch?v=q6fPk0--eHY

[8] *What is a wireless dead zone and how to eliminate it*, MakeUseOf. https://www.makeuseof.com/tag/what-is-a-wireless-dead-zone-and-how-to-eliminate-it/

[9] *Wi-Fi monitoring using Raspberry Pi*, Zone13. https://zone13.io/post/wifi-monitoring-using-raspberry-pi/

[10] P. Biondi *et al.*, *Scapy: Packet crafting for Python*. https://scapy.net

[11] *Kismet: wireless network detector, sniffer and WIDS*. https://www.kismetwireless.net

[12] Ministry of Civil Aviation (India), *Drone Rules 2021* & Digital Sky platform.

[13] U.S. FAA, *Remote Identification of Unmanned Aircraft*, Final Rule, 14 CFR Part 89.